

Frontend Handoff Document — Framework Call Tracker

1. Project Overview

The Framework Call Tracker is a frontend application for monitoring framework-wise client calls, their status, attempts, SLA performance, and call history.

The existing system has three important layers:

1. Google Sheets — source data
2. n8n — data processing/API layer
3. Frontend — dashboard and tracker UI

The frontend should NOT directly read Google Sheets. It should consume the n8n webhook APIs and render the returned data.

The current n8n workflow loads all function tabs and the `_CallLog`, processes framework calls, calculates SLA/verdict information, and returns JSON to the frontend.

2. Backend / API Base

Current n8n server:

```
https://srv1853541.hstgr.cloud
```

Webhook base:

```
https://srv1853541.hstgr.cloud/webhook/
```

Available endpoints:

GET — Main tracker data

```
/framework-tracker/data
```

Full:

```
https://srv1853541.hstgr.cloud/webhook/framework-tracker/data
```

This endpoint returns the processed framework-call dataset.

GET — Function / framework configuration

`/framework-tracker/functions`

Full:

`https://srv1853541.hstgr.cloud/webhook/framework-tracker/functions`

This endpoint is intended to provide the available function tabs/framework configuration. The n8n workflow has a dedicated `Webhook - Functions` → `HTTP - Sheet Tabs (Functions)` → `Filter Function Tabs` → `Respond - Functions` flow.

POST — Call status log

`/framework-tracker/log`

Full:

`https://srv1853541.hstgr.cloud/webhook/framework-tracker/log`

This receives status-change events and appends them to `_CallLog`.

3. Main Data Response

The `/framework-tracker/data` response is structured approximately as:

```
{
  "generatedAt": "ISO timestamp",
  "slaHours": 48,
  "counts": {
    "attempts": 0,
    "onTime": 0,
    "late": 0,
    "latePending": 0,
    "inWindow": 0,
    "paused": 0,
    "reopened": 0,
    "unmatched": 0
  },
  "rows": []
}
```

The n8n parser explicitly returns `generatedAt`, `slaHours`, `counts`, and `rows`.

4. Row Object

Each item inside `rows` contains:

```
{
  "business": "Business Name",
  "framework": "F1",
  "attempt": "A1",
  "advisor": "Advisor Name",
  "tab": "FUNCTION 1",
  "row": 2,
  "startDate": "2026-08-20",
  "dueDate": "2026-08-22",
  "calledDate": "2026-08-21",
  "currentStatus": "Completed",
  "outcome": "Completed",
  "verdict": "on_time",
  "lateHours": 0,
  "lateDays": 0,
  "reopened": false,
  "seededOnly": false,
  "history": []
}
```

The backend generates these fields directly, so the frontend should use them instead of recalculating SLA status itself.

5. Important Status Values

The tracker uses these statuses:

- No status
- Call pending
- Response pending
- Call postponed by client
- Call not received
- Completed

These are the canonical status values used by the backend.

Frontend should display these consistently.

Suggested UI treatment:

Status	Meaning
No status	No status has been recorded
Call pending	Call is currently pending
Response pending	Waiting for client response
Call postponed by client	Client postponed the call
Call not received	Call was not received
Completed	Call was successfully completed

Do not rename the values in the API layer.

6. Attempts

Each framework can have:

- A1
- A2

The backend reads and processes attempts individually.

Frontend should therefore treat:

F1 + A1

and

F1 + A2

as separate call records.

Do not combine attempts into one status.

7. Framework Structure

There are six framework groups/functions:

Function 1 — Team & People Leadership

4 frameworks:

- F1 — Hiring & Recruitment Framework
- F2 — Team KPI & Accountability Framework
- F3 — Leadership Development Framework

- F4 — Employee Growth & Retention Framework

Function 2 — Operation & Productivity Management

5 frameworks:

- F5 — SOP Writing Framework
- F6 — Department Structure Framework
- F7 — Daily Workflow Framework
- F8 — Productivity Tracking Framework
- F9 — Automation & Systemization Framework

Function 3 — Market Research & Product Strategy

4 frameworks:

- F10 — Customer Avatar & Buyer Psychology Framework
- F11 — Product Validation & Market Fit Framework
- F12 — Competitor Analysis & Market Positioning Framework
- F13 — Product Portfolio & Pricing Strategy Framework

Function 4 — Marketing & Lead Generation

4 frameworks:

- F14 — Brand Positioning Framework
- F15 — Content Strategy Framework
- F16 — Lead Generation Framework
- F17 — Marketing Funnel & Campaign Optimization Framework

Function 5 — Sales & Customer Conversion

6 frameworks:

- F18 — Sales Process Framework
- F19 — Sales Script & Objection Handling Framework
- F20 — CRM & Follow-up Framework
- F21 — Founder Role Clarity Framework
- F22 — Customer Conversion & Closing Framework
- F23 — Business Model Mapping Framework

Function 6 — Finance, Technology & Growth Management

6 frameworks:

- F24 — Profit Leakage Identification Model
- F25 — Business Reality Analysis Framework
- F26 — Cash Flow Management Framework

- F27 — Vision Clarity Framework
- F28 — Business Dashboard & KPI Technology Framework
- F29 — Business Growth & Scaling Framework

The framework names and ordering are defined in the existing Apps Script and must stay aligned with the n8n framework ordering.

8. SLA / Verdict Logic

The n8n backend uses a:

48-hour SLA

The backend calculates:

- `on_time`
- `late`
- `late_pending`
- `in_window`
- `paused`

It also tracks:

- `reopened`
- `unmatched`

The frontend should NOT independently calculate these values. Display the `verdict` returned by the API.

Recommended display

Backend verdict	Frontend label
<code>on_time</code>	On Time
<code>late</code>	Late
<code>late_pending</code>	Late — Pending
<code>in_window</code>	In Window
<code>paused</code>	Paused
<code>no_start_date</code>	No Start Date

Important: `no_start_date` can exist at row level even though it is not included as a separate count in the current `counts` object.

9. Dashboard KPIs

The main dashboard should expose the returned counts:

Primary KPI cards

1. Total Attempts
2. On Time
3. Late
4. Late Pending

Secondary metrics

- In Window
- Paused
- Reopened
- Unmatched

The backend already calculates these values, so frontend calculations should only be used for visual filtering or percentages.

10. Main Dashboard Table

Recommended columns:

Column	API field
Business	<code>business</code>
Advisor	<code>advisor</code>
Function	<code>tab</code> / <code>tabShort</code>
Framework	<code>framework</code>
Framework Name	<code>frameworkName</code> if available from the dashboard endpoint
Attempt	<code>attempt</code>
Start Date	<code>startDate</code>
Due Date	<code>dueDate</code>
Called Date	<code>calledDate</code>
Current Status	<code>currentStatus</code>
Outcome	<code>outcome</code>
SLA	<code>verdict</code>

Column	API field
Late	lateHours / lateDays

For the newer n8n response, use the `rows` schema. For the older Apps Script dashboard response, the equivalent call objects contain `tab`, `tabShort`, `row`, `advisor`, `business`, `framework`, `frameworkName`, `attempt`, `status`, `start`, `end`, `state`, `late`, and `gap`.

11. Filters

The frontend should provide:

Function filter

Examples:

- All Functions
- Function 1
- Function 2
- Function 3
- Function 4
- Function 5
- Function 6

Advisor filter

Populate dynamically from backend data.

Business filter

Populate dynamically from returned businesses.

Framework filter

F1–F29.

Attempt filter

- All
- A1
- A2

Status filter

Use the six canonical statuses.

Verdict filter

- All
- On Time
- Late
- Late Pending
- In Window
- Paused
- No Start Date

Filters should be frontend-only after the initial API response unless the backend later adds query parameters.

12. Business Detail View

Clicking a business should open a detail view.

Suggested layout:

Header

Business Name

Advisor Name

Function

Framework list

Each framework should show:

F1 – Hiring & Recruitment Framework

Then:

- A1
- A2

Each attempt should display:

- Start Date
- Due Date
- Called Date
- Current Status
- Outcome
- SLA result
- Late duration

History

If `history` exists, show the status timeline.

Each history item contains:

```
{
  "logId": 123,
  "date": "2026-08-25T10:30:00.000Z",
  "status": "Completed",
  "seeded": false
}
```

The n8n parser groups log events by business + framework + attempt and exposes this history.

13. Status Update / Logging

When a status changes, the frontend should POST to:

`/framework-tracker/log`

Expected payload:

```
{
  "editedAt": "2026-08-26T10:30:00.000Z",
  "advisor": "Advisor Name",
  "business": "Business Name",
  "changes": [
    {
      "framework": "F1",
      "attempt": "A1",
      "oldStatus": "Call pending",
      "newStatus": "Completed"
    }
  ]
}
```

The backend expects:

- `business`
- `changes`
- `editedAt`
- `advisor`

Each change should contain framework, attempt, and new status.

The backend converts the changes into `_CallLog` rows containing:

- logId
- date
- advisor
- business
- framework
- attempt
- status

This is handled by the n8n `Build Log Row` node.

14. Important Logging Rule

Do not send a log event just because the user opened a page.

Only send a log when a status actually changes.

The existing Apps Script takes a before/after snapshot and calculates the actual status differences before logging them.

Frontend equivalent:

```
Previous status: Call pending
New status:      Completed
                ↓
POST log event
```

If:

```
Previous status: Completed
New status:      Completed
```

Do not POST anything.

15. Data Refresh

The frontend should fetch fresh data when:

- Dashboard opens
- User clicks Refresh
- User returns to dashboard after changing a status

- A successful status update completes

Do not continuously poll unless specifically required.

The API response includes cache-control headers configured as no-cache/no-store, so the frontend can safely request fresh data.

16. Loading States

Every API request needs a proper loading state.

Recommended:

Initial load

Skeleton dashboard.

Refresh

Keep existing content visible and show a small loading indicator rather than blanking the whole page.

Status save

Disable the affected status control and show:

Saving...

After success:

Saved

After failure:

Could not save. Try again.

17. Error Handling

Backend errors can originate from:

- Google Sheets timeout
- Permission problems
- Invalid business
- Invalid framework
- Invalid payload
- Missing status
- Missing business

The existing Apps Script converts spreadsheet timeout and permission errors into user-friendly messages.

Frontend should never expose raw technical errors as the primary UI message.

Use:

```
Unable to load dashboard. Please try again.
```

or:

```
Unable to save this status. Please try again.
```

A detailed error can be logged to the console.

18. Recommended Frontend Architecture

Use a simple structure:

```
src/
├── api/
│   └── frameworkTracker.js
├── components/
│   ├── DashboardHeader
│   ├── KpiCard
│   ├── FilterBar
│   ├── StatusBadge
│   ├── VerdictBadge
│   ├── CallsTable
│   ├── BusinessDrawer
│   ├── FrameworkCard
│   ├── AttemptRow
│   └── CallHistory
├── pages/
│   ├── Dashboard
│   └── Tracker
├── hooks/
│   └── useFrameworkTracker.js
├── utils/
│   ├── formatDate.js
│   └── statusHelpers.js
```

|
└─ App.jsx

Keep API calls separate from UI components.

19. API Client

Create one API wrapper.

Example:

```
const API_BASE =
  'https://srv1853541.hstgr.cloud/webhook/framework-tracker';

export async function getTrackerData() {
  const response = await fetch(`${API_BASE}/data`, {
    method: 'GET',
    cache: 'no-store'
  });

  if (!response.ok) {
    throw new Error('Failed to load tracker data');
  }

  return response.json();
}

export async function getFunctions() {
  const response = await fetch(`${API_BASE}/functions`, {
    method: 'GET',
    cache: 'no-store'
  });

  if (!response.ok) {
    throw new Error('Failed to load functions');
  }

  return response.json();
}

export async function logStatusChange(payload) {
  const response = await fetch(`${API_BASE}/log`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
  },
```

```
    body: JSON.stringify(payload)
  });

  if (!response.ok) {
    throw new Error('Failed to save status change');
  }

  return response.json();
}
```

20. Do Not Duplicate Backend Business Logic

The frontend should not recreate:

- SLA calculations
- framework numbering
- framework grouping
- call outcome detection
- reopened detection
- log matching
- seeded-log handling
- deadline calculation

These are already handled by n8n.

The n8n parser uses a fixed framework ordering/count structure specifically so framework codes continue matching the source data and call logs.

21. Framework Numbering Warning

Framework numbering is important.

The source Apps Script supports both:

- per-tab numbering such as F1, F2
- continuous workbook numbering such as F5, F6 ... F29

The actual framework position is used to resolve the final framework code.

Frontend should therefore display the framework code received from the API instead of generating its own F-number.

22. Date Handling

API dates are ISO format:

YYYY-MM-DD

Example:

2026-08-26

Do not manually parse dates using locale-dependent JavaScript date strings.

Recommended display:

26 Aug 2026

Keep the original ISO value internally.

The existing system converts source spreadsheet dates into ISO dates before exposing them to the dashboard.

23. UX Requirements

The dashboard should be:

- Desktop-first but responsive
- Fast to scan
- Minimal
- Data-dense without feeling cluttered
- Clear about late calls
- Clear about pending calls
- Easy to filter
- Easy to identify problem areas

Priority hierarchy:

1. What needs attention?
2. Which business?
3. Which framework?
4. Which attempt?
5. How late?
6. What is the current status?
7. What happened previously?

24. Recommended Dashboard Layout

Framework Call Tracker	Refresh
Last updated: 26 Aug 2026	

Total Attempts		On Time	Late	Late Pending	
128		82	21	9	
Function	Advisor	Business	Status	Verdict	...
Function 1	Rahul	ABC Ltd	Completed	On Time	
Function 2	Amit	XYZ Ltd	Pending	Late	
Function 5	Neha	PQR Ltd	Response	Paused	

25. Status Colors

Suggested semantic colors:

- Completed → green
- Call pending → blue
- Response pending → purple/neutral
- Call postponed → amber
- Call not received → red
- No status → grey

For verdict:

- On Time → green
- In Window → blue
- Late → red
- Late Pending → red/strong warning
- Paused → neutral
- No Start Date → grey

Do not rely on color alone. Always include text labels.

26. Existing Google Apps Script UI Reference

There is an existing tracker UI that already establishes some interaction patterns:

- Function selector
- Business selector
- Advisor selector
- Searchable selectors
- Add-new-business action
- Add-new-advisor action
- Save button

- Loading/error states

The existing frontend loads configuration first and then enables the selectors.

The new frontend can improve the visual design while preserving these functional expectations.

27. Source of Truth

For frontend development:

Source data

Google Sheets

Processing

n8n

Frontend source

n8n JSON responses

Logging

n8n → `_CallLog`

The n8n workflow loads the source function tabs, separately loads `_CallLog`, combines the information, and produces the final processed dataset.

28. Frontend Development Checklist

- ☐ Create API client
- ☐ Connect `/data`
- ☐ Connect `/functions`
- ☐ Connect `/log`
- ☐ Build dashboard shell
- ☐ Build KPI cards
- ☐ Build filters
- ☐ Build calls table
- ☐ Build status badges
- ☐ Build verdict badges
- ☐ Build business detail view
- ☐ Build framework/attempt view
- ☐ Build call history
- ☐ Add loading states
- ☐ Add empty states

- ☐ Add API error states
- ☐ Add refresh action
- ☐ Add status-update flow
- ☐ Confirm only changed statuses are logged
- ☐ Test A1/A2 independently
- ☐ Test F1–F29 numbering
- ☐ Test late/on-time cases
- ☐ Test paused cases
- ☐ Test reopened cases
- ☐ Test unmatched log cases
- ☐ Test mobile responsiveness

29. Critical Things Not to Change

1. Do not change framework codes.
2. Do not rename backend status values.
3. Do not calculate SLA differently in frontend.
4. Do not write directly to Google Sheets from the frontend.
5. Do not create duplicate log entries for unchanged statuses.
6. Do not merge A1 and A2.
7. Do not assume every framework has the same number of frameworks per function.
8. Do not remove `history`.
9. Do not treat `late_pending` as the same thing as completed `late`.
10. Do not hardcode businesses or advisors.

30. Final Frontend Contract

The frontend developer should consider the n8n API the contract.

The frontend is responsible for:

Fetching → Filtering → Displaying → User interaction → Sending status changes

The backend is responsible for:

Reading Sheets → Joining logs → Calculating SLA → Determining verdict → Persisting logs

The current n8n workflow already returns the processed call rows and aggregate counts needed to build the dashboard.

The goal of the frontend is therefore to build a clean, responsive dashboard on top of this existing API rather than reproduce the backend logic.